

PROJET ANNUEL M2 – ESGI GRENoble

Business First

Plateforme E-Commerce SaaS Modulaire, Multi-Tenant & Microservices

Étudiants M. Seignovert • R. Verguet
• A. Da Silva

Filière Architecture des
Logiciels (AL)

Année
Académique
2025–2026

Présentation de l' Équipe & Rôles



Maxime Seignovert

Développeur Frontend & DevOps

Secteurs clés : Frontend React, intégration continue et infrastructure via la plateforme Coolify.

En charge du développement de l'interface utilisateur et de la configuration du déploiement applicatif.



Romain Verguet

Architecte Lead Backend

Secteurs clés : API Gateway (YARP), services métiers .NET et communication inter-services.

Garant du routage des requêtes système et du développement de l'architecture backend modulaire.



Alexy Da Silva

Expert DevOps & Backend

Secteurs clés : Conteneurisation globale, Docker Compose, services .NET et base de données.

Responsable de la mise en boîte via Docker et de la réalisation de différents services métiers robustes.

Le Besoin des Commerçants

Les artisans, PME et créateurs locaux font face à des freins majeurs lors du lancement de leur commerce en ligne :

-  **Dépendance & Rigidité** : Les solutions géantes imposent des modèles fermés et d'importantes barrières techniques.
-  **Coûts d'entrée élevés** : Abonnements lourds inadaptés aux micro-structures sans rentabilité immédiate.
-  **Complexité modulaire** : Difficulté à activer uniquement les outils requis (stocks, expédition, livraisons).



Les 3 Axes Structurants



Découpage Asynchrone

Éviter le couplage temporel. Chaque microservice métier interagit de manière autonome via un broker de messages robuste.



SaaS Multi-Tenant

Isolation absolue des données de chaque boutique au niveau SGBD via les politiques de sécurité PostgreSQL RLS.



Industrialisation

Conteneurisation globale, chaîne de CI/CD automatisée et déploiement simplifié sur VPS via la plateforme Coolify.

Gestion de Projet **Notion Kanban**

L'équipe utilise la méthodologie Agile avec des sprints de deux semaines. Le suivi des tâches en temps réel et la répartition s'effectuent via un tableau Kanban collaboratif sur Notion.



Kanban

The diagram illustrates a Kanban board with three columns representing different stages of task completion. Each column has a header with a colored circle and a count of tasks.

- Pas commencé (Not Started):** Represented by a grey circle and the number 4. It contains four task cards:
 - Système d'avis client:** Yellow star icon, 5 days remaining.
 - Moteur de recommandation:** Yellow star icon, 8 days remaining.
 - Service de création de facture:** Blue document icon.
 - Aide en ligne (ChatBot):** Blue robot icon.
- En cours (In Progress):** Represented by a blue circle and the number 2. It contains two task cards:
 - Service de notification:** Yellow bell icon.
 - Faire un service SignalR:** No icon.
- Terminé (Completed):** Represented by a green circle and the number 14. It contains four task cards:
 - Panel admin de gestion des stocks:** Red circle icon.
 - Faire la gateway:** No icon.
 - Implémenter le CRUD pour les produits dans ProductService:** No icon.
 - Faire le front mocké du crud articles:** No icon.

Each task card includes a title, an icon, a status indicator (A for Assigned, R for Reviewed), a name, and a time indicator (days remaining or completed).

Nouveau

Architecture Globale

Services découplés

Une architecture moderne et résiliente bâtie sur une passerelle .NET YARP unifiée.

Le frontend React communique avec une Gateway unique responsable de la validation JWT. Les services métiers (Product, Order, Payment, Inventory, Shipping) possèdent leur propre base de données distincte.

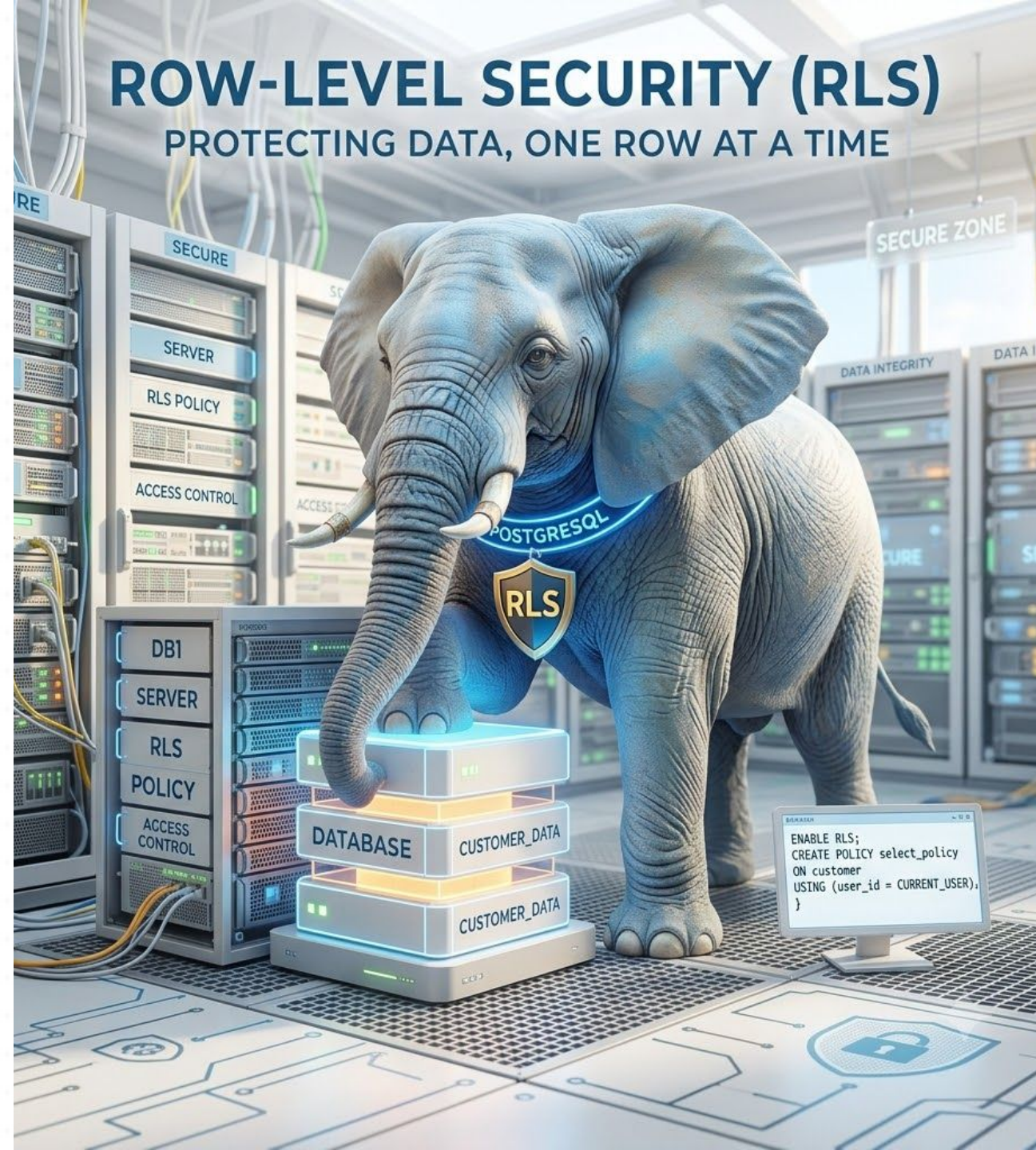


Isolation **Multi-Tenant**

Sécurité PostgreSQL native

Sécurité au plus bas niveau pour empêcher les fuites de données transverses entre boutiques.

En activant la Row-Level Security (RLS), chaque requête SQL est filtrée dynamiquement via le contexte global de session `app.current_tenant` injecté par l'API Gateway.



ROW-LEVEL SECURITY (RLS)

PROTECTING DATA, ONE ROW AT A TIME

```
ENABLE RLS;  
CREATE POLICY select_policy  
ON customer  
USING (user_id = CURRENT_USER);  
}
```

Messagerie Événementielle

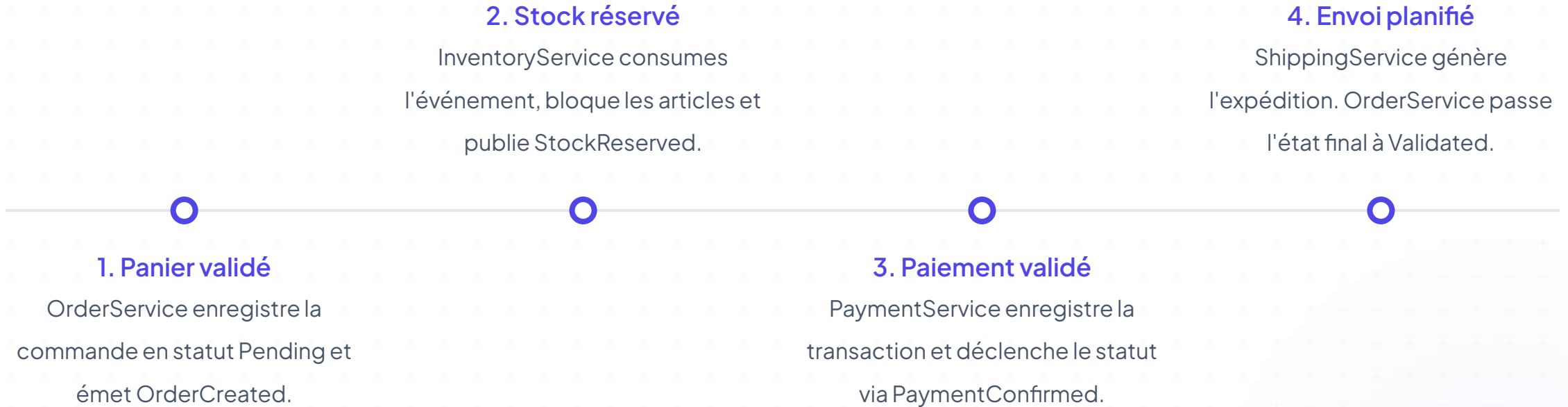
Le Choix de l'Asynchrone

Lorsqu'une commande est passée, l'utilisateur n'attend pas que le stock soit réservé. L'événement OrderCreated est publié de manière durable, garantissant un temps de réponse instantané et une forte tolérance aux pannes temporaires.

RabbitMQ vs Apache Kafka

RabbitMQ a été privilégié pour notre MVP SaaS. Il gère avec simplicité les files de messages conventionnelles, l'acquittement explicite et l'isolation des messages en échec durable (Dead-Letter Queue) sans la surcouche infra de Kafka.

Cycle de Commande Cible



Responsabilités des Services

Service	Rôle Fonctionnel Principal	Données Métier Clés	Actions Majeures
ProductService	Catalogue & gestion d'articles	Produits, catégories, prix, médias	Lecture Synchronne
OrderService	Cycle de vie de la commande	Commandes, lignes, statuts	Émetteur OrderCreated
InventoryService	Suivi des volumes et des seuils	Quantités physiques, réservations	Consomme OrderCreated
PaymentService	Facturation et traçabilité	Paiements, références Stripe	Consomme StockReserved
ShippingService	Logistique et expédition	Transporteurs, suivis colis	Consomme PaymentConfirmed

Gestion du Git via MR & Code Review



1. Merge Requests (MR)

Toute modification de code doit faire l'objet d'une branche dédiée et d'une demande de fusion (MR) vers la branche principale.



2. Revue par les Pairs

Au moins un autre développeur doit relire et valider les modifications pour assurer la qualité du code et le partage des connaissances.



3. Validation & CI/CD

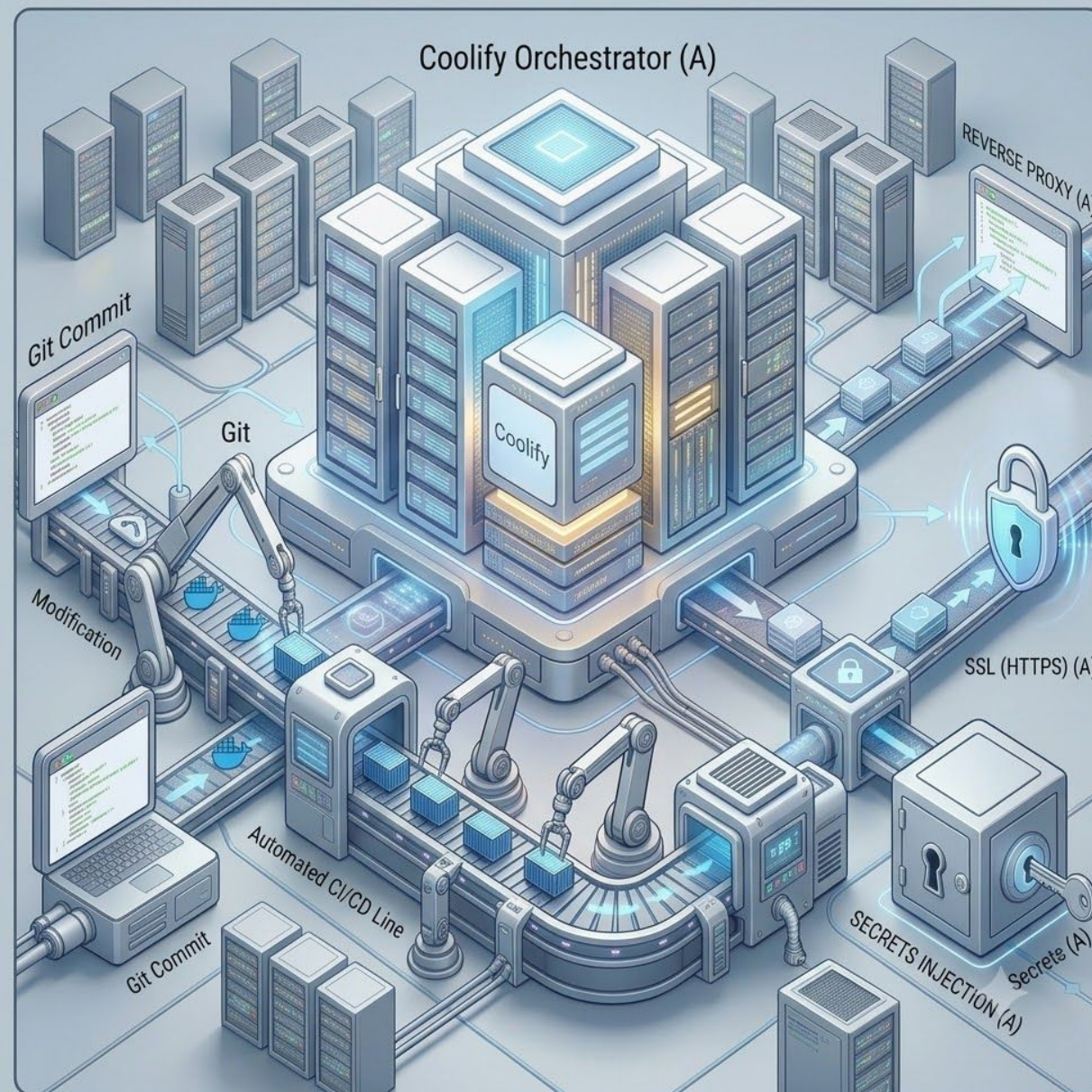
Le build automatique et les tests doivent passer avec succès avant que la fusion ne soit autorisée vers la branche de production.

Déploiement Continu

CI/CD & DevOps Industrialisé

Chaque modification poussée sur Git déclenche automatiquement notre pipeline GitHub Actions.

Les images Docker sont construites et livrées directement à un VPS géré par Coolify. Coolify agit comme orchestrateur léger, gérant le reverse proxy, le SSL (HTTPS) et l'injection des secrets.



Stratégie globale de **Validation**

-  **Tests Unitaires & Règles Métier** : Isolation des services .NET pour valider de manière exhaustive les calculs de taxes, prix et seuils d'inventaire.
-  **Tests Multi-Tenant d'Isolation** : Tentatives d'injection transversales (forger un identifiant de Tenant B depuis le compte A). PostgreSQL bloque automatiquement l'accès via les règles RLS.
-  **Tests de Résilience** : Simulation d'indisponibilité du service d'inventaire. La commande reste dans la file d'attente RabbitMQ et se rejoue dès sa reconnexion.
-  **Pipeline de Livraison** : Automatisation complète du build Docker et des étapes de linting sur chaque Pull Request pour assurer une qualité constante.

Limites & Perspectives

Limites Identifiées

-  **Complexité réseau** : Besoin d'un outillage d'observabilité fort (type OpenTelemetry).
-  **Échelle Multi-Tenant** : Base partagée idéale au lancement, mais limite d'isolation physique pour de très grands comptes.

Évolutions Futures

-  **Analytics Module** : Pour suivre la conversion et les performances des boutiques.
-  **Intégration Stripe** : Pour un encaissement de production.
-  **Kubernetes** : À envisager lors d'une forte montée en charge commerciale du produit.

Des Questions ?

Merci pour votre attention. Place à la démonstration.